BÀI THỰC HÀNH BUILDER PATTERN (FLUENT – KHÔNG DIRECTOR)

CÔNG NGHỆ SỬ DỤNG

✅ Backend: ASP.NET Core Web API
✅ Frontend: React (gọi API)
✅ Áp dụng Builder Pattern chuẩn GoF (Fluent Builder)

BÀI TOÁN

Hệ thống **E-commerce** cho phép tạo **Order linh hoạt**:

Order có thể bao gồm:

- Nhiều Product
- Có Tax
- Có Shipping
- Có Discount

👉 User có thể chọn **tùy ý các thành phần**

Ví dụ:

Scenario	Thành phần
Basic	Product
Custom 1	Product + Tax
Custom 2	Product + Shipping
Custom 3	Product + Tax + Discount

👉 Nếu dùng constructor:

❌ Rất phức tạp
❌ Khó mở rộng

👉 Giải pháp:

👉 **Builder Pattern (Fluent – không Director)**

KIẾN TRÚC HỆ THỐNG

```
Frontend (React)
        ↓
POST /api/orders/build
        ↓
OrdersController
        ↓
OrderBuilder (Fluent)
        ↓
Order (Result)
```

🔷 BACKEND – ASP.NET CORE WEB API

1️⃣ TẠO PROJECT

```
dotnet new webapi -n BuilderDemo
cd BuilderDemo
```

CẤU TRÚC THƯ MỤC

```
BuilderDemo
├── Controllers
│     └── OrdersController.cs
│
├── Builders
│     └── OrderBuilder.cs
│
├── Models
│     ├── Order.cs
│     └── BuildOrderRequest.cs
│
└── Program.cs
```

---

1. MODEL ORDER

Models/Order.cs

```
namespace BuilderDemo.Models;

public class Order
{
    public List<string> Products { get; set; } = new();

    public decimal Tax { get; set; }
    public decimal Shipping { get; set; }
    public decimal Discount { get; set; }

    public decimal GetTotal()
    {
        var productTotal = Products.Count * 100;

        return productTotal + Tax + Shipping - Discount;
    }
}
```

---

2. BUILDER (FLUENT)

Builders/OrderBuilder.cs

```
using BuilderDemo.Models;

namespace BuilderDemo.Builders;

public class OrderBuilder
{
    private Order _order = new();

    public OrderBuilder AddProduct(string product)
    {
        _order.Products.Add(product);
        return this;
    }

    public OrderBuilder AddTax()
    {
        _order.Tax = 10;
```

```
        return this;
    }

    public OrderBuilder AddShipping()
    {
        _order.Shipping = 5;
        return this;
    }

    public OrderBuilder AddDiscount()
    {
        _order.Discount = 20;
        return this;
    }

    public Order Build()
    {
        return _order;
    }
}
```

---

3. REQUEST MODEL

Models/BuildOrderRequest.cs

```
namespace BuilderDemo.Models;

public class BuildOrderRequest
{
    public List<string> Products { get; set; }

    public bool HasTax { get; set; }
    public bool HasShipping { get; set; }
    public bool HasDiscount { get; set; }
}
```

---

4. CONTROLLER

Controllers/OrdersController.cs

```
using Microsoft.AspNetCore.Mvc;
using BuilderDemo.Builders;
```

```
using BuilderDemo.Models;

namespace BuilderDemo.Controllers;

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    [HttpPost("build")]
    public IActionResult BuildOrder(BuildOrderRequest
request)
    {
        var builder = new OrderBuilder();

        // Add products
        foreach (var product in request.Products)
        {
            builder.AddProduct(product);
        }

        // Runtime options
        if (request.HasTax)
            builder.AddTax();

        if (request.HasShipping)
            builder.AddShipping();

        if (request.HasDiscount)
            builder.AddDiscount();

        var order = builder.Build();

        return Ok(new
        {
            order.Products,
            order.Tax,
            order.Shipping,
            order.Discount,
            Total = order.GetTotal()
        });
    }
}
```

---

5. PROGRAM

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Add services
builder.Services.AddControllers();

// Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// CORS
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact",
        policy =>
        {
            policy.WithOrigins("http://localhost:3000")
                  .AllowAnyHeader()
                  .AllowAnyMethod();
        });
});

var app = builder.Build();

// Swagger
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.UseCors("AllowReact");

app.MapControllers();

app.Run();
```

CHẠY BACKEND

```
dotnet add package Swashbuckle.AspNetCore

dotnet build
dotnet run
```

Swagger:

```
https://localhost:xxxx/swagger
```

FRONTEND – REACT

---

1 TẠO PROJECT

```
npx create-react-app builder-ui
cd builder-ui
npm install axios
```

---

2 GIAO DIỆN BUILD ORDER

src/App.js

```
import React, { useState } from "react";
import axios from "axios";

function App() {

  const [products, setProducts] = useState(["Laptop"]);
  const [tax, setTax] = useState(false);
  const [shipping, setShipping] = useState(false);
  const [discount, setDiscount] = useState(false);
  const [result, setResult] = useState(null);

  const buildOrder = async () => {

    const response = await axios.post(
      "https://localhost:xxxx/api/orders/build",
      {
        products: products,
```

```
        hasTax: tax,
        hasShipping: shipping,
        hasDiscount: discount
      }
    );

    setResult(response.data);
  };

  return (
    <div style={{ padding: 40 }}>

      <h2>Build Order</h2>

      <h4>Options</h4>

      <label>
        <input type="checkbox"
          onChange={(e) => setTax(e.target.checked)} />
        Tax
      </label>

      <br />

      <label>
        <input type="checkbox"
          onChange={(e) => setShipping(e.target.checked)}
/>
        Shipping
      </label>

      <br />

      <label>
        <input type="checkbox"
          onChange={(e) => setDiscount(e.target.checked)}
/>
        Discount
      </label>

      <br /><br />

      <button onClick={buildOrder}>
        Build Order
```

```
      </button>

      <h3>Result:</h3>

      {result && (
        <div>
          <p><b>Products:</b> {result.products.join(",
")}</p>
          <p><b>Tax:</b> {result.tax}</p>
          <p><b>Shipping:</b> {result.shipping}</p>
          <p><b>Discount:</b> {result.discount}</p>
          <p><b>Total:</b> {result.total}</p>
        </div>
      )}

    </div>
  );
}

export default App;
```

CHẠY FRONTEND

```
npm start
```

🔷 LUỒNG XỬ LÝ

KHI USER CHỌN OPTION

```
1 User chọn:
   ✔ Tax
   ✔ Shipping

2 React gửi request:

{
  "products": ["Laptop"],
  "hasTax": true,
  "hasShipping": true,
  "hasDiscount": false
}
```

```
3 Controller gọi Builder:

builder.AddProduct()
builder.AddTax()
builder.AddShipping()

4 Build()

5 Trả kết quả
```

🔷 KIẾN THỨC SINH VIÊN ĐẠT ĐƯỢC

Sau bài này sinh viên hiểu rõ:

1️⃣ Builder Pattern

Xây dựng object từng bước

```
builder
    .AddProduct()
    .AddTax()
    .AddShipping()
    .Build();
```

2️⃣ Fluent Builder

- Method chaining
- Runtime configuration

3️⃣ So sánh với Director

Có Director	Không Director
Flow cố định	Linh hoạt runtime
Ít dùng thực tế	Dùng nhiều trong production

🔷 BÀI TẬP VỀ NHÀ

👉 Phân tích:

Hiện tại Controller đang trực tiếp tạo:

```
var builder = new OrderBuilder();
```

👉 Điều này gây **tight coupling**

YÊU CẦU NÂNG CẤP

👉 Hãy cải tiến hệ thống bằng cách:

- Sử dụng **Dependency Injection**
- Inject `OrderBuilder` vào Controller
- Tách logic sang Service